

01 — PRISM Architecture

Audience: engineers, architects, anyone who needs the whole shape of the system in one sitting.

Companion docs: [02 Deployment](#) · [03 Module interaction](#) · [06 Code map](#)

1. What PRISM is

PRISM is Evam Finance's climate-finance origination and servicing platform. It carries a deal from the first conversation an RM has in the field, through qualification, credit approval, syndication or asset monetisation, to disbursement and loan servicing — and it keeps an auditable record of every decision along the way.

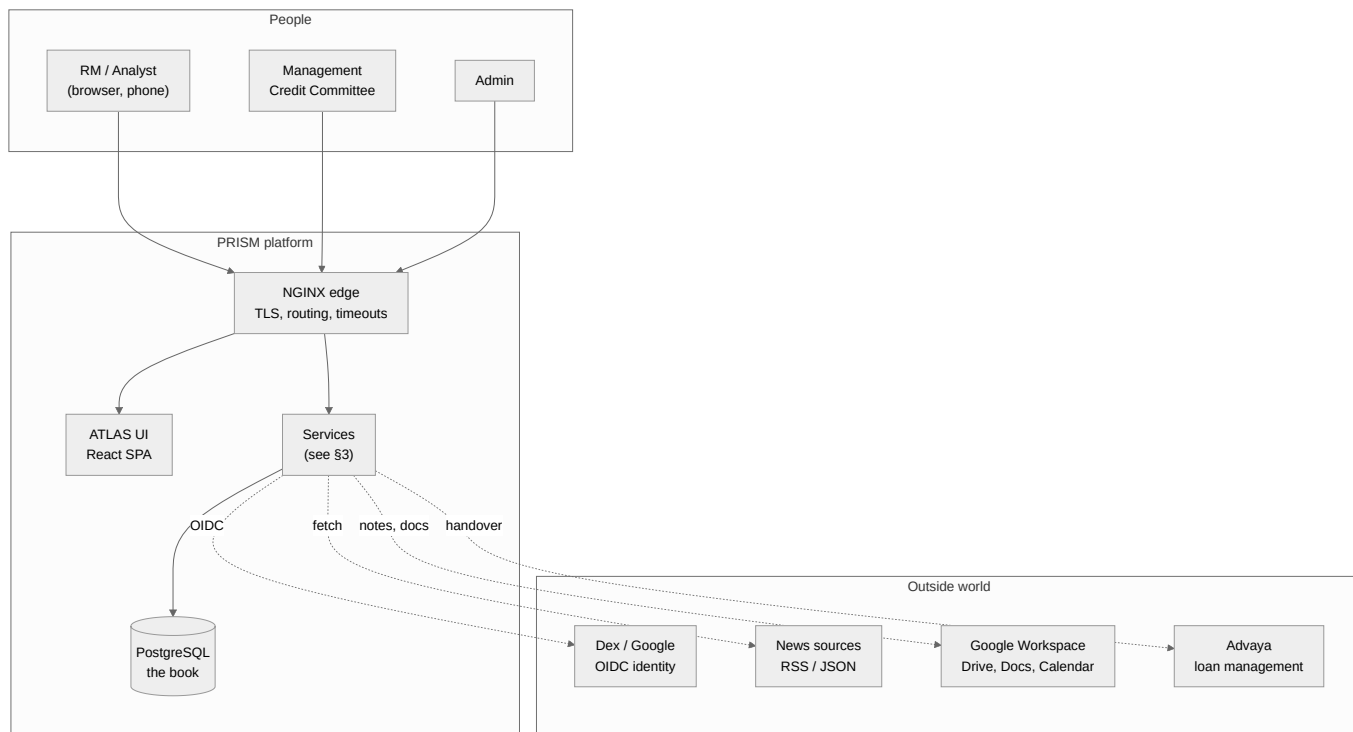
The product surface people actually use is called **ATLAS** (the web dashboard) and **VocX** (voice capture). Everything else is infrastructure behind those two.

The three design commitments

Every structural choice in this codebase follows from three commitments. If a piece of the system looks odd, it is almost always one of these being honoured.

Commitment	What it means in practice
The Register is the book.	Exactly one service owns business data. Every other service is stateless and reads/writes through the Register's API. A bug in the dashboard, the news radar, or a workflow can never corrupt the record.
Authorization is decided once, at the edge, and carried.	The gateway verifies identity, resolves permissions from the Access service, and mints a <i>signed</i> internal context. Downstream services enforce from that signature — they never re-derive authority from a stale local copy of the matrix.
Anything that must not be half-done is a workflow.	Multi-step business processes with human decision points (lead conversion, sanction, handover) run as durable Temporal workflows with explicit compensation, not as a chain of HTTP calls that can die mid-way.

2. System context



Boundaries worth naming. Dex (or Google directly) is the only identity authority — PRISM stores no passwords. News sources are pulled, never pushed. The Advaya handover is one-way and gated by an explicit money-movement approval. Google Workspace integration is per-user OAuth, and its tokens live outside the repository (see [12 VocX & STT](#)).

3. Service catalogue

Ten runtime services plus two shared libraries. Each is a directory under `services/` or `packages/`.

Service	Port (compose)	Owns data?	One-line purpose
register	8000	Yes — the only one	The book. Entities, leads, deals, all three product trackers, loan accounts, documents, decisions, audit.
gateway	8001	No	The single front door. Verifies OIDC, resolves permissions, mints the signed internal context, routes to every backend.
access	8002	Yes (identity only)	Users, role grants, the live RBAC matrix. Answers "what may this person do?"
atlas	8005	No	Read-side BFF. Composes dashboard, "Today", pipeline and company-summary payloads.
vocx	8003	No (files only)	Voice capture → transcript → structured touchpoint → Register write. Owns audio archive + Google tokens on disk.
stt	internal	No	Speech-to-text. faster-whisper on CPU, behind an OpenAI-compatible endpoint.
pulse	8004	No	News / adverse-media radar. Fetch, match, classify RED/AMBER/GREEN, file intel idempotently.
workflows (worker)	—	No	Temporal worker. Executes every durable business workflow.
orchestrator	8006	No	HTTP front for Temporal: starts workflows, delivers signals (approve/reject).
notifier	—	No	Drains the notification outbox and delivers (email/SMTP).
ui	80	No	Static NGINX serving the built ATLAS React bundle.

Supporting infrastructure: **postgres** (16), **minio** (S3-compatible object store for documents), **temporal** + **temporal-ui**, **dex** (OIDC, `sso` profile), **nginx** (edge), **pgbbackup** (`backup` profile).

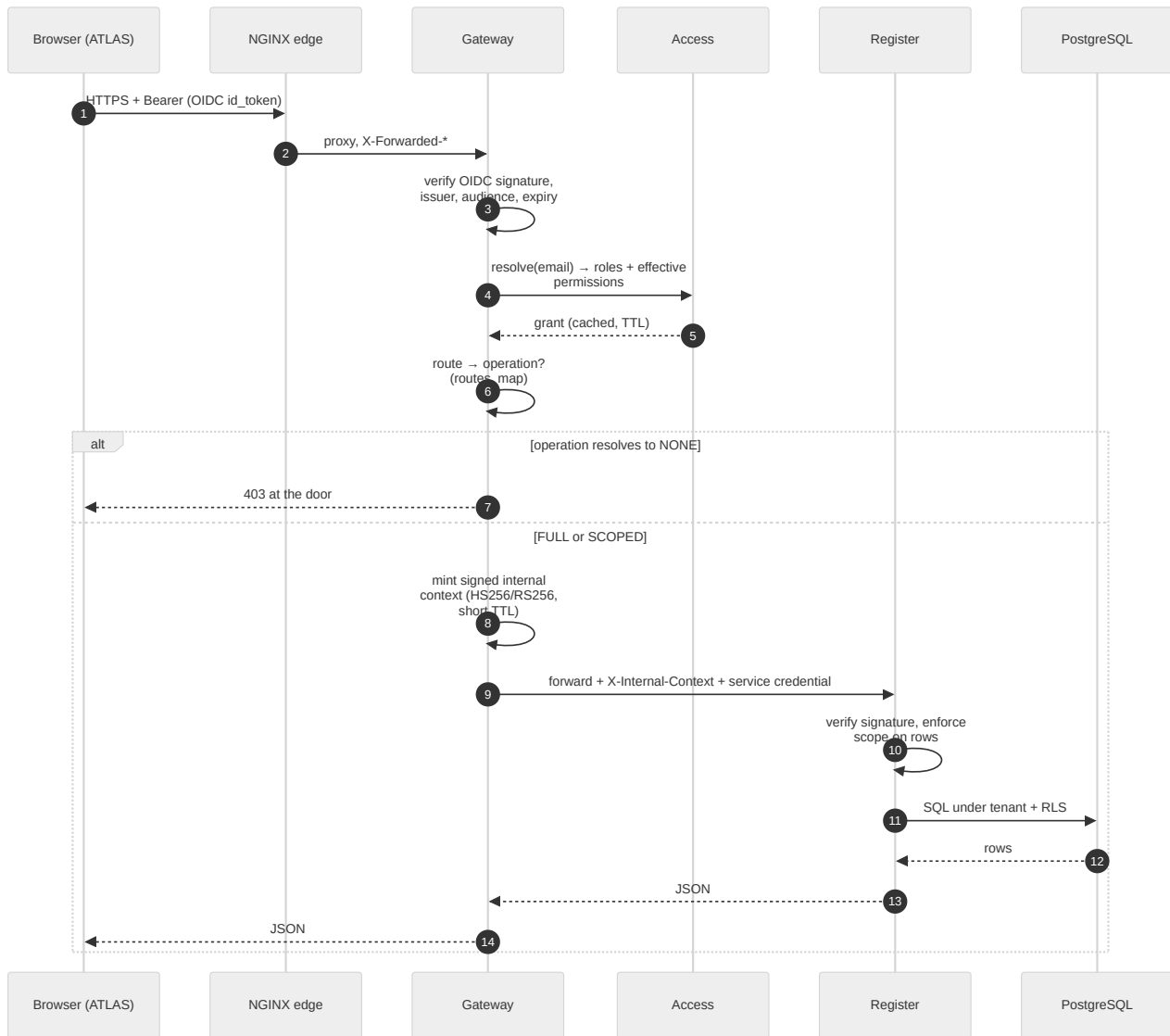
Shared libraries

Package	What lives there
<code>packages/evam-backend-core</code>	The spine every FastAPI service imports: app factory, config, DB session, CRUD, pagination, errors, logging, health, RBAC catalogue , policy/scope evaluator , signed internal token , service principal policy , retry, lifecycle.
<code>packages/evam-register-client</code>	Typed HTTP client for the Register, used by ATLAS, PULSE, VocX and the workflow activities. Nothing else should hand-roll Register calls.

The rule this enforces: **cross-cutting security lives in one library, not copied into ten services**. If you find authorization logic re-implemented in a service, that is a bug.

4. The request path

Every browser request follows the same spine. This is the diagram to hold in your head.



Three properties fall out of this shape:

1. **A request rejected at the gateway never touches the database.** Cheap denial.
2. **A request that gets through still gets checked at the Register.** Defence in depth — the gateway's route map is an accelerator, not the only gate. A route absent from `routes_map.py` forwards and is enforced downstream, which is the safe default.
3. **Nothing downstream trusts a plaintext header.** `X-User-Email` from the outside is stripped at the gateway; identity only ever arrives as a signature.

Store	Owner	Backed up by
PostgreSQL (business + identity)	register, access	<code>pgbackup</code> service + <code>prism-deploy.sh backup</code>
MinIO object store (documents)	register	volume snapshot (see 09)
VocX audio archive	vocx	volume snapshot
VocX Google OAuth tokens	vocx	secrets snapshot only — never in the repo
Temporal history	temporal	its own PostgreSQL database

7. Where each concern is implemented

A short index so you can jump straight to code. The full version is [06 Code map](#).

Concern	Where
Role list, tiers, aliases	<code>packages/evam-backend-core/evam_backend_core/rbac_catalog.py</code>
Scope evaluation ("may this user write <i>this row</i> ")	<code>packages/evam-backend-core/evam_backend_core/policy.py</code>
Signed internal context	<code>packages/evam-backend-core/evam_backend_core/internal_token.py</code>
Machine capability allowlists	<code>packages/evam-backend-core/evam_backend_core/service_policy.py</code>
Route → operation map (edge gate)	<code>services/gateway/app/routes_map.py</code>
Tables	<code>services/register/app/models/*.py</code>
REST surface	<code>services/register/app/api/*.py</code>
Lifecycle vocabulary + legal stage moves	<code>services/register/app/api/</code> + <code>services/atlas/ui/src/**/*.lifecycle</code>
Durable business processes	<code>services/workflows/app/workflows.py</code>
Register calls made by workflows	<code>services/workflows/app/activities.py</code>
Front-end RBAC mirror	<code>services/atlas/ui/src/auth/rbac.ts</code>
Edge timeouts, body size, upstreams	<code>deploy/nginx/nginx.conf</code>

8. Technology choices, and why

Choice	Reason
FastAPI + SQLAlchemy (async)	One idiom across every Python service; OpenAPI generated from the same types the code enforces.
PostgreSQL, self-hosted	Row-level security, strong constraints, and no cloud lock-in for a book of record. See docs/adr/0005-postgresql-self-hosted.md .
Row-level security (RLS), fail-closed	Tenant isolation enforced by the database, not only by application <code>WHERE</code> clauses. A missing filter is a bug that RLS still contains.
Optimistic concurrency (version column)	Two RMs editing the same deal produce a visible 409, never a silent lost update. See docs/adr/0003-optimistic-concurrency.md .
Temporal	Business processes that span days and human approvals need durable state, retries and compensation. An HTTP chain cannot survive a redeploy mid-approval.
Monorepo with shared core	Security invariants live in one library. See docs/adr/0004-monorepo-shared-core.md .
React + MUI SPA	One dashboard used on desktop and phone; the desk works from a phone in the field.
faster-whisper on CPU	Voice capture must work on the EC2 box the platform already runs on, with no GPU. Accuracy tuned by beam size and thread count.

9. What is deliberately *not* here

Naming the absences saves the next engineer a search.

- **No message bus.** Cross-service async work goes through Temporal or the notification outbox table, not Kafka/RabbitMQ.
- **No service mesh.** Trust is carried in a signed token, not by mTLS between pods.
- **No separate read replica / CQRS store.** ATLAS composes reads live from the Register and caches briefly; the book is small enough that this is honest and simple.
- **No secrets manager in the compose deployment.** Secrets are files on disk under `deploy/`, excluded from git, snapshotted by the deploy script. Helm deployments use Kubernetes Secrets.
- **No front-end test suite.** The ATLAS UI is verified by TypeScript, by browser-driven checks during development, and by the E2E runbooks under `docs/`. This is a known gap.

10. Reading order from here

If you are...	Read next
A new backend engineer	06 Code map → 08 Register → 15 Data model
A new front-end engineer	11 ATLAS usage → 06 Code map → 07 RBAC
On call / deploying	02 Deployment → 10 Upgrade → 13 Operations
Wiring a new business process	05 Temporal → 04 Running flows
Doing a security review	07 RBAC → 03 Module interaction → §5 above